Plane Sweep Algorithm Implementation

GPU Computing: Architectural Trade-offs and Performance Analysis

Niccolò Benedetto Paolo Alberto Bordis

Polytech - Université Libre de Bruxelles



June 2026

Instructors: Prof. Daniele Bonatto, Eline Soeten

Introduction

- **Objective**: Implement and critically evaluate the Plane Sweep algorithm on high-performance GPU architectures.
- **Methodology**: Progressively time and memory optimization guided by the analysis of the main bottlenecks
- **Source Code**: `https://github.com/dreakess/PlaneSweep505`

Hardware Specifications

Table: NVIDIA GeForce GTX 1080 Ti Architectural Parameters

Specification	**Value**
Compute Capability	6.1
Max Threads per block	1024
Multiprocessors (SM)	28
Single Precision (FP32)	3914.99 GFLOP/s
Global Memory Bandwidth	484.44 GB/s
L2 Cache Size	2816 KB
Global Memory Size	10.99 GB

Existing algorithm: Plane Sweep

- **Assumption**: 3D space is partitioned into parallel planes.
- **Projection**: The reference image is projected onto each plane hypothesis.
- **Evaluation**: A cost function evaluates the similarity between the reference and secondary views.

Sum of Absolute Differences (SAD)

$$SAD = \sum_{i,j \in W} |I_{ref}(i,j) - I_{sec}(i,j)|$$

This approach suits quite well for cuda programming since the problem is embarassengly parallel

Cost Volume Regularization

- **The Cost Volume**: The raw output of the SAD computation contains matching costs.
- **Inherent Limitations**: Local matching estimates are highly susceptible to noise in textureless regions or occluded boundaries.
- **Refinement (Graph Cut)**:
  - Acts as a global optimization technique.
  - Enforces spatial consistency across the depth map.
  - Crucial for generating sharp, mathematically sound depth discontinuities.

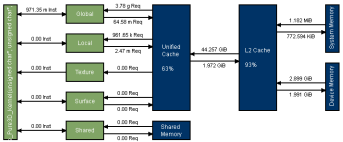
Implementation Strategy I: Naive GPU

- **Approach**: Direct port of the CPU algorithm to CUDA kernels.
- **Topology**: Block size $[16, 16]$, Grid size

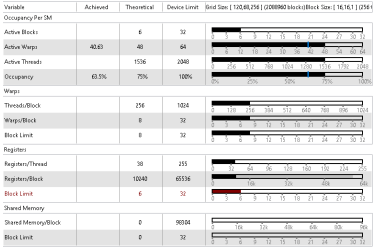
$$[\frac{width + BLOCK_SIZE - 1}{BLOCK_SIZE}, \frac{height + BLOCK_SIZE - 1}{BLOCK_SIZE}, ZPlanes]$$

  matching the cost cube.
- **Execution**: Core SAD computation resides inside the kernel, launched independently for each camera.
- **Initial Outcome**: Achieves a baseline 151x speedup over CPU, but suffers from suboptimal memory access patterns.

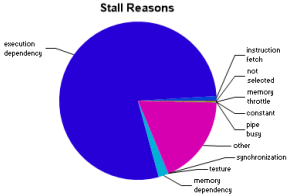
Naive GPU: Profiler Analysis



(a) Memory Statistics



(b) Compute Resources



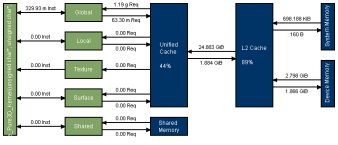
(c) Warp State (Stalls)

- **Memory:** Large traffic between L1 and L2 and high number of requests
- **Resources:** Low occupancy and high register usage limit the number of blocks that can run concurrently on each SM
- **states:** Warp spend most of their time waiting for previous ALU instructions to complete

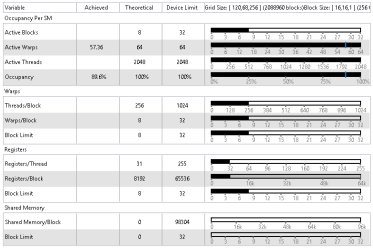
Implementation Strategy II: Naive Optimized

- **Data Type Alignment**: Transitioned from `double` to `float` to exploit the GPU's native FP32 throughput.
- **Constant Memory Utilization**: Static camera parameters were relocated to Constant Memory.
- **Architectural Impact**:
  - Leverages constant caching and broadcasting mechanisms.
  - Drastically reduces the load/store burden on the L2 cache.

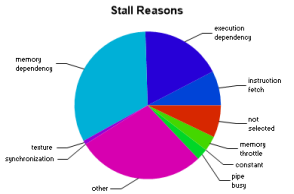
Naive Optimized: Profiler Analysis



(a) Memory Statistics



(b) Compute Resources



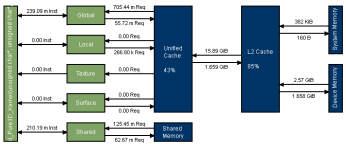
(c) Warp State (Stalls)

- **Memory:** Reduced traffic between caches
- **Resources:** Achieved high occupancy and reduction of number of registers per block
- **States:** Warps spend most of their time waiting for doing a request of load/store to complete

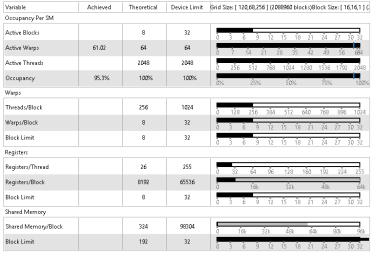
Implementation Strategy III: Shared Memory

- **Approach**: Pre-loading the entire reference camera data block into Shared Memory.
- **Theoretical Benefit**: Minimization of repetitive global memory reads.
- **Practical Cost**: Introduces strict thread synchronization requirements (`__syncthreads()`).

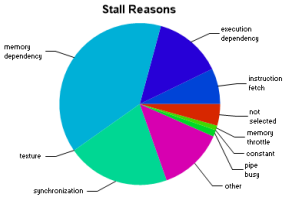
Shared Memory: Profiler Analysis



(a) Memory Statistics



(b) Compute Resources



(c) Warp State (Stalls)

- **Memory:** High request between kernel and shared memory, therefore reducing traffic between L1 and L2
- **Resources:** Achieved the highest occupancy
- **States:** Warp also spend time for synchronization

Performance Benchmark ($RAD = 1$)

Table: Resolution: 1920×1080, 256 depth planes

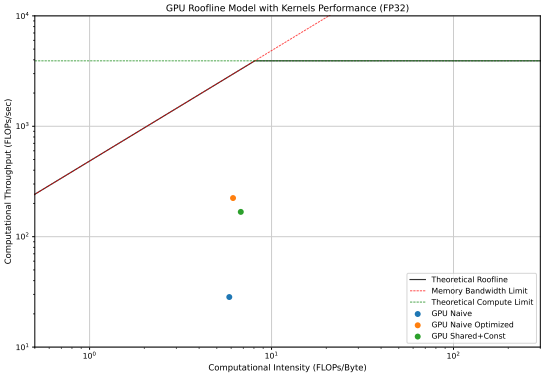
Algorithm & Strategy	**Kernel Time (s)**	**Speedup**	**GFLOP/s**
Naive CPU	149.0690	1.00x	0.53
Naive GPU	0.9904	151.00x	28.40
Naive Optimized GPU	**0.1280**	**1164.00x**	**223.85**
Shared Memory GPU	0.1709	872.00x	167.72

Analysis: The Naive Optimized version statistically outperforms the Shared Memory variant. The synchronization penalty imposed by shared memory effectively neutralizes the bandwidth savings at this computational intensity.

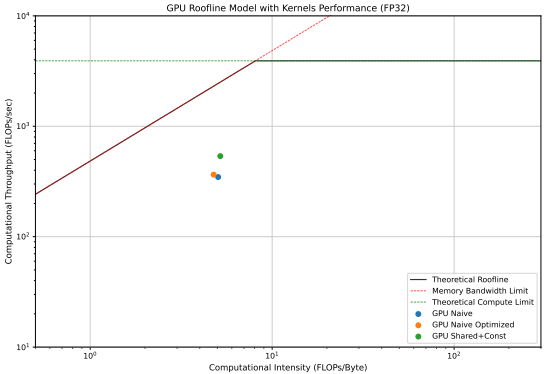
Evaluating the Hardware Limit: The Roofline Model

- To objectively interpret why Shared Memory underperformed at $RAD = 1$, we must evaluate the **Arithmetic Intensity** against theoretical limits.
- The Roofline model explain whether a kernel is **Memory-Bound** or **Compute-Bound**.
- **Experiment**: We artificially increased the workload radius to $RAD = 11$ to exploit the advantage of the shared usage.

Roofline Evaluation: Workload Sensitivity



(a) Low Intensity: $RAD = 1$



(b) High Intensity: $RAD = 11$

- **Conclusion**: At $RAD = 11$, the kernel becomes strictly Compute-Bound. **Shared Memory** implementation overperforms the Naive optimization

Conclusions & Future Work

- **Hardware Truths**: Manual memory management (Shared Memory) is not universally beneficial. It requires sufficient computational intensity to offset synchronization penalties.
- **L2 Cache Efficacy**: Modern GPUs (like the GTX 1080 Ti) exploits highly optimized L2 caches in order to address light workloads ($RAD = 1$).
- **Future Stress Testing**:
  - Scaling input arrays to 2K/4K resolutions.
  - Expanding the multi-view framework beyond two secondary cameras.